# JavaScript Interview Concepts

1. Hoisting
2. Closures
3. Var, Let, Const & Temporal Dead Zone
4. This keyword
5. Callback Functions
6. null and undefined
7. Promises, Promise APIs, Promise Chaining
8. Async and Await
9. Call, Apply, Bind
10. Event loop in js
11. Arrow function and regular function
12. Shallow and Deep copy
13. Prototype and prototypal inheritance
14. First class function
15. Anonymous Functions
16. Higher order function
17. Scopes
18. ES6
19. OOPS in js
20. Currying in js

## Hoisting

Hoisting is a concept that enables us to extract the values of variables and functions even before initialization or assignment without encountering an error. This occurs due to the first phase (memory creation phase) of the Execution Context. In this initial phase, the JavaScript engine allocates memory to all variables and functions. For variables, it assigns the value undefined, and for functions, it copies the entire function body.

## Closures

A closure is a function that has access to its outer function scope even after the function has returned. Meaning, A closure can remember and access variables and arguments reference of its outer function even after the function has returned.

So, Lexical Environment = local memory + lexical env of its parent. Hence, Lexical Environment is the local memory along with the lexical environment of its parent.

Whenever an Execution Context is created, a Lexical environment(LE) is also created and is referenced in the local Execution Context(in memory space).

## Variable Declarations

### Var

- **Scope:** Function-scoped
- **Re-declaration:** ✅ Allowed
- **Re-assignment:** ✅ Allowed
- **Hoisted:** Initialized to undefined
- **Initial Value:** Optional

### Let

- **Introduced in:** ES6
- **Scope:** Block-scoped
- **Re-declaration:** ❌ Not allowed
- **Re-assignment:** ✅ Allowed
- **Hoisted:** Yes (but in TDZ)
- **Initial value:** Optional

### Const

- **Introduced in:** ES6
- **Scope:** Block-scoped
- **Re-declaration:** ❌ Not allowed
- **Re-assignment:** ❌ Not allowed
- **Hoisted:** Yes (but in TDZ)
- **Initial value:** Required

## Temporal Dead Zone (TDZ)

Time since when the let and const variable was hoisted until it is initialized some value.

## This Keyword

In JavaScript, `this` is a keyword that always refers to an object:

- The value of `this` is dynamically determined

- When used outside function and object, `this` refers to the global object
- When used inside the method of object, `this` refers to the object itself
- When used inside function, `this` refers to the object that the function is called on

## Callback Functions

A callback function is a function passed as an argument to another function and is executed later, usually after some operation is completed.

### Example:

```javascript
function greet(name, callback) {
    console.log("Hello, " + name);
    callback();
}

function sayBye() {
    console.log("Goodbye!");
}

greet("Rajat", sayBye);
```

### Why Use Callback Functions?

- To execute code after a task is complete (e.g., after data is loaded)
- They are essential for asynchronous programming in JavaScript

## Undefined vs Not Defined in JS

- **undefined:** When memory is allocated for the variable, but no value is assigned yet
- **not defined:** When the variable itself is not declared but called in code

## Promises

A Promise is an object that represents the eventual completion (or failure) of an asynchronous operation and its resulting value.

- Introduced in ES6
- Promises are an advancement over callback functions
- They help prevent our code from falling into "callback hell"

### States of a Promise

- **Pending:** Initial state, not fulfilled or rejected

- **Fulfilled:** Operation completed successfully
- **Rejected:** Operation failed

## Promise APIs

1. Promise.all
2. Promise.allSettled
3. Promise.race
4. Promise.any

### Promise.all

Takes a list of promises and returns one new promise.

- That new promise succeeds only if all the given promises succeed
- If any one promise fails, the whole thing fails immediately with that error
- If all succeed, you get an array of results

```javascript
const p1 = Promise.resolve(1);
const p2 = Promise.resolve(2);
const p3 = Promise.resolve(3);

Promise.all([p1, p2, p3])
  .then(results => console.log(results)) // [1, 2, 3]
  .catch(error => console.log(error));
```

### Promise.allSettled() — Wait for Everything, Good or Bad

- Takes a list of promises and returns one new promise
- Waits until all promises are either fulfilled or rejected
- Returns a list with each promise's status and value or error
- It never fails, even if some promises fail

### Promise.race() — "Whoever Finishes First Wins"

- Takes a list of promises
- The returned promise settles as soon as one promise settles (resolves or rejects)
- Returns the value or error of the first one that finishes

### Promise.any() — "First One to Succeed"

- Takes a list of promises

- Resolves when the first promise is fulfilled
- If all promises fail, it rejects with an AggregateError

### Promise Chaining

Means linking multiple `.then()` calls one after another, where each step uses the result of the previous one. It helps to run asynchronous tasks in a sequence, instead of nesting callbacks.

## Async and Await

- Async Await allows to write async code in sync manner
- **Async** keyword is used to define a function that returns a promise
- **Await** keyword is used to pause the execution until Promise is resolved or rejected
- Async Await is built on top of Promises
- Async Await introduced in ES2017

## Call, Apply, Bind

All three methods — `call()`, `apply()`, and `bind()` — are used to manually set the value of `this` when calling a function.

### call()

- Calls the function immediately
- Accepts arguments one by one
- Returns function result

```javascript
function greet(city) {
  console.log(`Hello, I'm ${this.name} from ${city}`);
}

const person = { name: "Rajat" };
greet.call(person, "Indore");
```

### apply()

- Also calls the function immediately
- Accepts arguments as an array
- Returns the function result

```javascript
greet.apply(person, ["Indore"]);
```

### bind()

- Does not call the function immediately
- Returns a new function with `this` bound
- You can call it later

```javascript
const greetRajat = greet.bind(person, "Indore");
greetRajat();
```

## Event Loop

It is a mechanism that allows JavaScript to handle async operations such as timers, callbacks and non-blocking operations in an efficient way.

- The Event loop constantly monitors the call stack and callback queue
- If the call stack is empty, the event loop will move a callback function from callback queue to the call stack to be executed
- Once the callback function has executed, it's popped off the call stack and event loop continues to check callbacks in the callback queue
- This process keeps repeating, allowing JS to handle async tasks without blocking the main thread

### Call Stack

A data structure that keeps track of currently executing functions in JS. When a function is called, it's pushed onto the call stack and when it's finished, it's popped off the stack.

### Microtask Queue

- Microtask is same as callback queue but it has higher priority
- Functions in Microtask queue are executed early
- All the callback functions that come through promises go in microtask queue

## Arrow Function vs Regular Function

### Arrow Function

1. Arrow function is also known as fat arrow function
2. Arrow function introduced in ES6

3. Arrow Function provides shorter syntax
4. Arrow functions don't have their own `this` concept. The value of `this` is determined by surrounding lexical context
5. Arrow Functions don't have arguments object
6. Arrow functions cannot be used as a constructor
7. Arrow functions are not fully hoisted

### Regular Function

1. Can be used as a Constructor
2. Have their own `this`
3. Have arguments object
4. Regular functions are fully hoisted

## Shallow and Deep Copy

In JS, when we copy one object into another object, it will copy the reference of the object.

There are two ways to copy object by value in JS:

### 1. Shallow Copy

A shallow copy copies only the top-level properties. If the object has nested objects, only the reference is copied — not the actual data.

There are two ways to perform shallow copy in JS:

```javascript
let user = Object.assign({}, original);
const shallowCopy = { ...original };
```

### 2. Deep Copy

A deep copy copies everything, including all nested levels. The copied object is completely independent.

```javascript
const deepCopy = JSON.parse(JSON.stringify(original));
```

## Prototype and Prototypal Inheritance

### Prototype

Every object in JavaScript has a hidden property called `[[Prototype]]` (you can access it using `__proto__` or `Object.getPrototypeOf()`). This prototype is another object that the current object can inherit properties and methods from.

```javascript
const person = {
  greet() {
    console.log(`Hi, I'm ${this.name}`);
  }
};

const user = {
  name: "Rajat"
};

user.__proto__ = person;
user.greet(); // Hi, I'm Rajat
```

`user` doesn't have its own `greet()`, so JavaScript looks up the prototype chain and finds it in `person`.

### Prototypal Inheritance

Prototypal inheritance means one object can inherit directly from another object.

```javascript
function User(name) {
  this.name = name;
}

User.prototype.sayHello = function () {
  console.log(`Hello, ${this.name}`);
};

const u1 = new User("Rajat");
u1.sayHello(); // Hello, Rajat ✅
```

## First Class Function

In JavaScript, functions are treated like values — they can be:

- Assigned to variables
- Passed as arguments
- Returned from other functions

This makes them first-class citizens in the language.

## Anonymous Functions

An anonymous function is a function without a name.

Used mostly:

- In callbacks
- When functions are passed as arguments

```javascript
setTimeout(function () {
    console.log("Hello after 1 sec");
}, 1000);
```

## Higher-Order Functions (HOF)

A Higher-Order Function is a function that:

✅ Takes one or more functions as arguments **OR** ✅ Returns a function

### Common HOFs in JavaScript:

- `map()`
- `filter()`
- `reduce()`
- Custom HOFs you write

## Scopes in JavaScript

In JavaScript, scope determines where variables are accessible in your code.

**Scope = Where a variable is visible.**

### 1. Global Scope

Variables declared outside any function or block. Accessible anywhere in the file.

### 2. Function Scope

Variables declared inside a function are only accessible within that function.

### 3. Block Scope (let and const)

Variables declared with `let` or `const` inside `{}` are not accessible outside.

### 4. Lexical Scope

Functions can access variables from their outer scope where they were defined.

## ES6 Features

### Template Literals

Supports multi-line strings and interpolation.

### Default Parameters

### Destructuring (Arrays & Objects)

```javascript
const [a, b] = [1, 2]; // Array destructuring
const { name, age } = { name: "Rajat", age: 25 }; // Object destructuring
```

### Rest and Spread Operators

### Import/Export

### For-of Loop vs For-in Loop

#### For-of

- Introduced in ES6
- Designed for iterating over iterable objects like arrays
- No need for index handling

#### For-in

- Designed for iterating over the properties of objects

## OOP (Object-Oriented Programming) in JavaScript

JavaScript supports Object-Oriented Programming (OOP), even though it's prototype-based (not class-based like Java or C++).

### 1. Class & Object

A class is a blueprint. An object is an instance of a class.

### 2. Encapsulation

javascript

```javascript
class BankAccount {
    #balance = 0; // 🔒 private field

    deposit(amount) {
        this.#balance += amount;
    }

    getBalance() {
        return this.#balance;
    }
}

const account = new BankAccount();
account.deposit(1000);
console.log(account.getBalance());
```

### 3. Abstraction — (Hide Complex Logic Behind Simple Interface)

javascript

```javascript
class Car {
    startEngine() {
        this.#injectFuel();
        this.#ignite();
        console.log("Engine started");
    }

    #injectFuel() { /* hidden */ }
    #ignite() { /* hidden */ }
}

const myCar = new Car();
myCar.startEngine(); // ✅
```

## Currying

Currying is a technique in JavaScript where a function doesn't take all arguments at once, but instead takes one argument at a time and returns a new function for the next argument.

## Modules

In JavaScript, modules are reusable pieces of code that are encapsulated in their own scope and can be imported/exported between files. They help organize code, avoid naming conflicts, and improve maintainability.

### Why Use Modules?

- Code reusability
- Modularity
- Better maintainability

## Array Methods

### Slice

1. Slice is used to extract substring from string
2. Slice does not modify original array
3. Returns new extracted elements

### Splice

- ❌ Modifies the original array
- ✅ Can add or remove elements
- Returns deleted elements

## Module Systems

### module.exports

Is used in CommonJS (the module system used in Node.js) to export functions, objects, or variables from a module so they can be used in another file.

## Object Methods

### Object.seal()

- Prevents adding new properties
- Prevents removing existing properties
- ✅ You can still update existing properties

### Object.freeze()

- Prevents adding new properties
- Prevents deleting properties
- ❌ Prevents modifying existing property values